Amazon Web Services

Complete Learning Guide

Beginner-Friendly · In-Depth · Practical

A comprehensive reference covering all major AWS Analytics services — from data ingestion to visualization — with real-world use cases, architecture diagrams, comparison tables, and hands-on examples.

Services Covered

#	Service	Core Purpose
1	**Amazon EMR**	Big Data Processing at Scale
2	**AWS Lake Formation**	Build Secure Data Lakes Faster
3	**Amazon Redshift**	Cloud Data Warehousing
4	**Amazon Kinesis**	Real-Time Data Streaming
5	**AWS Glue**	Serverless ETL & Data Catalog
6	**Amazon MSK**	Managed Apache Kafka
7	**Amazon OpenSearch Service**	Search & Log Analytics
8	**Amazon QuickSight**	Business Intelligence & Visualization
9	**Amazon Athena**	Serverless Interactive Query

Version 1.0 · AWS Certified Training Material

Table of Contents

Service 1

Amazon EMR

Elastic MapReduce — Big Data Processing at Cloud Scale

1.1 Simple Introduction

What is Amazon EMR?

Amazon EMR (Elastic MapReduce) is a fully managed cloud service that makes it easy to process vast amounts of data quickly and cost-effectively using open-source frameworks such as Apache Hadoop, Apache Spark, Apache Hive, Apache HBase, Apache Flink, Apache Hudi, and Presto. AWS handles the provisioning, configuration, and management of the underlying infrastructure, so you can focus on running your analytics jobs.

Why Does EMR Exist?

Before cloud computing, companies that needed to process terabytes or petabytes of data had to buy and maintain expensive on-premises clusters. Setting up Hadoop or Spark clusters took weeks of engineering time, and the hardware sat idle most of the time. EMR was created to eliminate this overhead — spin up a 100-node cluster in minutes, run your job, then shut it down, paying only for what you use.

Real-World Analogy

■ **EMR is like a temporary factory.** Imagine you run a food processing company and need to process 1 million tons of raw ingredients before the holiday season. Instead of buying a permanent factory (expensive, idle most of the year), you rent a massive factory for just 3 weeks, hire workers, process everything, then close the factory. EMR works exactly the same way — you rent computing power (EC2 instances), process your big data, then terminate the cluster.

Business Problem it Solves

Problem	How EMR Solves It
Processing TBs/PBs of raw data daily	Distributed processing across 100s of nodes in parallel
High cost of on-premises Hadoop clusters	Pay-per-use; terminate when done, saving 60-80%
Complex cluster setup and maintenance	Fully managed; AWS handles patching, config, scaling
Slow ML model training on large datasets	Spark MLlib on EMR handles distributed ML training
ETL pipelines that take hours	Parallelism across many nodes cuts ETL time dramatically

1.2 Core Concepts & Terminology

Cluster

An EMR **Cluster** is a collection of EC2 instances (called nodes) that work together to process data. A cluster has three types of nodes:

Node Type	Role	Example Instances
Master Node	Manages the cluster, coordinates tasks, tracks job status. Single point of control.	m5.xlarge, m5.2xlarge
Core Node	Runs tasks AND stores data in HDFS (Hadoop Distributed File System). Required.	r5.2xlarge, r5.4xlarge
Task Node	Runs tasks ONLY — no data storage. Optional. Good for spot instances.	c5.4xlarge (spot)

Key Terminology

Term	Definition
HDFS	Hadoop Distributed File System — distributes file blocks across Core Nodes for redundancy and fast parallel reads.
EMRFS	EMR File System — lets EMR use S3 as a persistent storage layer (recommended over HDFS for durability).
Step	A unit of work submitted to a cluster (e.g., a Spark job, a Hive query). Steps execute sequentially by default.
Bootstrap Action	Script that runs on all nodes before the cluster starts. Used to install software or configure settings.
Application	Open-source framework installed on EMR: Spark, Hadoop, Hive, HBase, Flink, Presto, etc.
Instance Fleet	A mix of instance types and purchase options (On-Demand + Spot) to optimize cost and availability.
Auto Scaling	Automatically add/remove Task Nodes based on YARN memory or custom CloudWatch metrics.
Kerberos	Authentication protocol available on EMR for securing cluster access in enterprise environments.

1.3 Architecture & Working

High-Level Architecture

```
User / Application
↓ Submit Job (Spark/Hive/Pig script via Console, CLI, SDK, Step Functions)
EMR Control Plane (AWS Managed)
↓ Provisions EC2 instances, installs frameworks, configures networking
Master Node (ResourceManager / NameNode / Driver)
↓ Distributes tasks to workers
Core Nodes (NodeManager + DataNode + Executor)
↓ Read input data from S3 or HDFS
Task Nodes (NodeManager + Executor — no HDFS)
↓ Process partitions in parallel
```

```
Output → S3 / Redshift / DynamoDB / HBase
```

Step-by-Step Workflow

- **Step 1 — Launch Cluster:** You specify instance types, count, applications (Spark, Hive…), and the cluster spins up in 5–10 minutes. EMR installs frameworks automatically.
- **Step 2 — Data Ingestion:** Input data resides in Amazon S3 (preferred) or HDFS. EMRFS provides direct S3 reads with consistency.
- **Step 3 — Job Submission:** A Spark job or Hive query is submitted as a *Step*. The Master Node's ResourceManager (YARN) schedules tasks across all available NodeManagers.
- **Step 4 — Distributed Execution:** Each Core/Task Node processes a subset of the data in parallel. Spark breaks the job into stages → tasks. Shuffle operations redistribute data between stages.
- **Step 5 — Output Writing:** Results are written back to S3, Redshift (via JDBC), or another sink.
- **Step 6 — Cluster Termination:** For transient clusters, after all steps complete, EMR terminates the cluster automatically. You pay only for the runtime.

Cluster Types

Type	Description	Best For
Transient Cluster	Spin up → Run job → Terminate automatically	Batch ETL, scheduled pipelines
Long-Running Cluster	Cluster stays up indefinitely. You terminate manually.	Interactive analysis, persistent Hive metastore
EMR Serverless	No cluster management — just submit jobs; AWS auto-scales	Sporadic workloads, simplified ops (newer option)
EMR on EKS	Run EMR workloads on Amazon EKS (Kubernetes)	Teams already using Kubernetes for orchestration

1.4 Major Features

Feature	Description	Business Value
Spot Instance Integration	Use EC2 Spot Instances for Task Nodes (up to 90% cheaper than On-Demand)	Dramatic cost reduction for fault-tolerant batch jobs
Auto Scaling	Scale Task Nodes in/out based on YARN or CloudWatch metrics	Handle variable workloads without over-provisioning
EMR Studio	Browser-based IDE (Jupyter notebooks) to develop and debug Spark/Hive jobs	Faster developer onboarding; collaborative notebooks
Managed Scaling	AWS automatically adjusts cluster size for optimal cost/performance	Hands-off optimization for production clusters
Multiple Frameworks	Spark, Hadoop, Hive, HBase, Flink, Presto, Livy — all on one cluster	Use the right tool for each workload
Encryption	At-rest (S3-SSE, HDFS) and in-transit (TLS) encryption	Compliance with HIPAA, PCI-DSS, GDPR

Feature	Description	Business Value
Lake Formation Integration	Fine-grained column and row-level access control via Lake Formation	Data governance in multi-team environments
Graviton Instances	Run EMR on ARM-based Graviton EC2 instances (up to 15% faster, 10% cheaper)	Better price-performance for Spark workloads

1.5 Real-World Use Cases

Industry	Use Case	Frameworks Used
E-Commerce (Amazon)	Process billions of clickstream events nightly to update recommendation models	Spark, Hive, Hadoop
Banking	Run regulatory risk calculations (VaR, stress tests) across millions of financial instruments	Spark, custom Java MapReduce
Healthcare	Genomics — align and analyze whole-genome sequencing data (100GB+ per patient)	Spark, GATK on EMR
Media & Entertainment	Transcode millions of video files; generate thumbnails; extract metadata	Hadoop MapReduce, Spark
Advertising	Compute real-time bidding features — aggregate ad impressions and clicks hourly	Spark Streaming, Flink
Retail Forecasting	Train demand forecasting models on 5 years of daily sales data (TBs)	Spark MLlib, XGBoost on EMR
Social Media	Analyze social graph (billions of user connections) for fraud/spam patterns	GraphX on Spark

1.6 Practical Example — PySpark ETL on EMR

Scenario: Aggregate daily sales data from S3 and write results back to S3

The following PySpark script reads raw CSV sales data, computes total revenue per product category, and writes the aggregated Parquet output to an S3 output bucket. This would be submitted as an EMR Step.

```
# daily_sales_etl.py
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum as _sum

spark = SparkSession.builder.appName('DailySalesETL').getOrCreate()

# Read raw CSV from S3
df = spark.read.option('header','true').option('inferSchema','true')
.csv('s3://my-raw-bucket/sales/2024/01/01/')

# Transform: aggregate revenue by product category
result = (df.groupBy('product_category')
.agg(_sum(col('quantity') * col('unit_price')).alias('total_revenue'))
.orderBy('total_revenue', ascending=False))

# Write Parquet output to S3
result.write.mode('overwrite').parquet('s3://my-output-bucket/sales-agg/2024/01/01/')
spark.stop()
```

```
# Submit as EMR Step via AWS CLI:
aws emr add-steps --cluster-id j-XXXXXXXXXXXX --steps
'[{"Name":"DailySalesETL","ActionOnFailure":"CONTINUE",
"HadoopJarStep":{"Jar":"command-runner.jar",
"Args":["spark-submit","s3://my-scripts/daily_sales_etl.py"]}}]'
```

1.7 Integrations & Comparison

Key AWS Integrations

AWS Service	How it Integrates with EMR
Amazon S3	Primary storage — EMR reads input and writes output via EMRFS. Durable, scalable, cheap.
AWS Glue Data Catalog	EMR uses Glue Catalog as its Hive Metastore — shared schema across Athena, Glue, EMR.
Amazon Redshift	EMR jobs can write directly to Redshift via JDBC or use Redshift Spectrum for querying.
Amazon DynamoDB	EMR can read/write DynamoDB tables using the DynamoDB Hadoop connector.
Amazon SageMaker	Train ML models on EMR Spark, then deploy with SageMaker. Or use SageMaker Processing jobs on EMR.
AWS Step Functions	Orchestrate multi-step EMR pipelines (create cluster → add steps → terminate → notify).
Amazon CloudWatch	Monitor cluster metrics (YARN memory, node failures). Trigger Auto Scaling rules.
AWS Lake Formation	Column-level and row-level access control on EMR queries via LF-Tags.

Comparison: EMR vs Glue vs Athena

Dimension	Amazon EMR	AWS Glue	Amazon Athena
Type	Managed cluster service	Serverless ETL service	Serverless query service
Framework	Spark, Hadoop, Hive, Flink, Presto	Spark (Glue ETL), Python Shell	Presto / Trino SQL
Control	Full control over cluster config	Abstracted — just write ETL code	Zero infrastructure
Cost Model	Per EC2 instance-hour + EMR fee	Per DPU-hour (Data Processing Unit)	Per TB of data scanned
Best For	Complex pipelines, ML, stream processing	Simpler ETL, catalog-driven pipelines	Ad-hoc SQL on S3
Startup Time	5–10 min (cluster launch)	~1 min (warm containers)	Seconds (serverless)
Skill Required	Spark/Hadoop expertise	Python / Spark basics	SQL

Service 2

AWS Lake Formation

Build, Secure, and Manage Data Lakes in Days, Not Months

2.1 Simple Introduction

What is AWS Lake Formation?

AWS Lake Formation is a fully managed service that helps you set up a secure, centralized data lake on Amazon S3 in days rather than months. It automates the complex, manual steps typically required to collect, cleanse, catalog, and secure large amounts of data from diverse sources. Once your data lake is set up, analytics services like Athena, Redshift Spectrum, and EMR can immediately query the data with fine-grained security controls.

Real-World Analogy

■■ **Lake Formation is like a modern library system.** A raw data lake is like a warehouse full of books dumped in random piles — unusable. Lake Formation acts as the library management system: it catalogs every book (Data Catalog), decides who can access which section (access control), and automatically restocks shelves with new arrivals (blueprints/crawlers). Librarians (data engineers) can find and share data instantly instead of searching through chaos.

Business Problem it Solves

Challenge Without Lake Formation	How Lake Formation Helps
Months of manual work to set up a data lake	Automated ingestion via Blueprints and Crawlers in hours
Data is siloed across S3 buckets, RDS, on-prem	Centralized catalog with unified metadata
S3 bucket policies are coarse-grained (all or nothing)	Column-level and row-level security via LF-Tags
Different teams using different schema definitions	Single Glue Data Catalog shared across all analytics services
Impossible to audit who accessed which data	CloudTrail + Lake Formation access logs for full auditability
Cross-account data sharing is complex	Lake Formation RAM (Resource Access Manager) integration

2.2 Core Concepts & Terminology

Concept	Definition
Data Lake	Centralized repository on S3 storing structured, semi-structured, and unstructured data at any scale.
Data Catalog	AWS Glue Data Catalog — metadata repository storing table definitions, schemas, and data locations.
Blueprint	Pre-built templates for ingesting data from sources (S3, RDS, DynamoDB) into the data lake automatically.
Workflow	A blueprint instantiation — the actual data ingestion job that crawls, catalogs, and loads data.

Concept	Definition
LF-Tags (Lake Formation Tags)	Attribute-based access control (ABAC) system — tag databases/tables/columns with key-value pairs, then grant access by tag.
Data Permissions	Fine-grained permissions on Catalog databases, tables, and columns for IAM principals.
Governed Tables	Lake Formation–managed tables with ACID transactions, automatic data compaction, and granular access control.
Data Lake Administrator	IAM user/role designated as the admin of the data lake — can grant/revoke all Lake Formation permissions.
Cross-Account Access	Share Catalog resources (databases, tables) with other AWS accounts using AWS RAM without copying data.
Row Filter	Lake Formation feature to restrict which rows a user can query (row-level security).

2.3 Architecture & Working

```
DATA SOURCES
S3 buckets ■ RDS / Aurora ■ DynamoDB ■ On-Premises (via DataSync/DMS)
↓ [Blueprints / AWS Glue Crawlers]
INGESTION LAYER
Raw Zone (S3) → Glue ETL Jobs → Curated Zone (S3 Parquet)
↓ [Glue Data Catalog registration]
LAKE FORMATION CONTROL PLANE
Metadata Management ■ LF-Tag Policies ■ Permission Engine
↓ [Permission check on every query]
CONSUMERS (Query Engines)
Amazon Athena ■ Redshift Spectrum ■ Amazon EMR ■ SageMaker
↓
OUTPUT: Dashboards ■ ML Models ■ Reports ■ APIs
```

How Fine-Grained Access Works

- **Step 1:** Data Lake Admin defines LF-Tags: e.g., sensitivity=HIGH, dept=Finance.
- **Step 2:** Tags are assigned to Catalog resources: databases, tables, or individual columns.
- **Step 3:** IAM principals (users/roles) are granted access based on tags, not S3 bucket policies.
- **Step 4:** When an analyst queries a table via Athena, Lake Formation checks their tag-based permissions.
- **Step 5:** Lake Formation filters out unauthorized columns/rows before returning results — transparently.

2.4 Features

Feature	Description
Blueprints	One-click data ingestion pipelines from 17+ source types. Automates Glue crawling and ETL.
LF-Tag Based Access Control	ABAC model — manage permissions at scale without writing individual resource-level policies.

Feature	Description
Column-Level Security	Restrict access to specific columns (e.g., hide SSN, salary from non-HR users).
Row-Level Filtering	Filter rows returned based on the querying user's attributes (e.g., region = 'US').
Data Sharing (Cross-Account)	Share databases/tables with other AWS accounts. Consumer queries data in-place without copying.
Governed Tables	S3-backed tables with ACID transactions — safe concurrent writes without data corruption.
Automatic Data Compaction	Lake Formation compacts small files in Governed Tables automatically, improving query performance.
Audit Logging	All data access events logged to CloudTrail for compliance and security auditing.

2.5 Real-World Use Cases

Scenario	Details
Healthcare — Patient Data Governance	Hospital groups create a data lake. Lake Formation ensures doctors see only their patients' data (row-level). Billing sees cost columns only.
Financial Services — Multi-Dept Analytics	Bank's Risk, Compliance, and Marketing teams share one data lake but see different subsets via LF-Tags.
Retail — Supplier Data Sharing	Retailer shares sales analytics tables with suppliers (cross-account) without exposing raw PII.
Media — Content Metadata Lake	Video platform centralizes metadata from 50+ content systems. Analysts query via Athena without knowing source systems.
Government — Data Democratization	Federal agency builds a lake with strict row-level filters ensuring state-level analysts only see their state's data.

2.6 Practical Example — Setting Up a Data Lake

Here's a step-by-step walkthrough using the AWS CLI to register an S3 location, create a database, and grant column-level access:

```
# 1. Register S3 location with Lake Formation
aws lakeformation register-resource
--resource-arn arn:aws:s3:::my-data-lake-bucket
--use-service-linked-role

# 2. Create Glue database (appears in LF catalog)
aws glue create-database --database-input '{"Name": "sales_db"}'

# 3. Grant SELECT on specific columns to analyst role
aws lakeformation grant-permissions
--principal DataLakePrincipalIdentifier=arn:aws:iam::123456789:role/AnalystRole
--resource '{"TableWithColumns":{"DatabaseName":"sales_db",
"Name":"transactions",
"ColumnNames":["order_id","amount","date"]}}'
--permissions SELECT
```

2.7 Integrations & Comparison

AWS Service	Integration Role
AWS Glue	Lake Formation uses Glue Data Catalog as its metadata store. Glue Crawlers populate the catalog.
Amazon S3	Physical storage for the data lake. Lake Formation manages access policies on top of S3.
Amazon Athena	Queries Glue Catalog tables governed by Lake Formation — column/row filters applied automatically.
Amazon Redshift Spectrum	Queries external tables in the Glue Catalog with Lake Formation permission enforcement.
Amazon EMR	EMR clusters can be configured to use Lake Formation for fine-grained access on Hive/Spark queries.
AWS IAM	Lake Formation permissions complement IAM policies — both must allow access for a query to succeed.

Lake Formation vs Raw S3 + IAM

Dimension	Raw S3 + IAM Policies	AWS Lake Formation
Access Granularity	Bucket or prefix level	Column-level and row-level
Management Overhead	High — manage per-bucket policies	Low — centralized tag-based policies
Audit	S3 access logs (limited)	Detailed CloudTrail per table/column access
Cross-Account Sharing	Complex — requires bucket policies + IAM	Simple — RAM-based sharing in minutes
ETL Automation	Manual	Blueprints automate ingestion pipelines

Service 3

Amazon Redshift

Petabyte-Scale Cloud Data Warehousing with MPP Architecture

3.1 Simple Introduction

What is Amazon Redshift?

Amazon Redshift is a fully managed, petabyte-scale cloud data warehouse service. It is designed for Online Analytical Processing (OLAP) workloads — running complex SQL queries across billions of rows at high speed. Redshift uses a Massively Parallel Processing (MPP) architecture, columnar storage, and advanced compression to deliver fast query performance at a fraction of the cost of traditional data warehouse appliances like Teradata or Netezza.

Real-World Analogy

■ **Redshift is like a supercharged accounting department.** A bank has millions of transactions flowing through each day. Regular databases (like MySQL) are like one accountant with a ledger — fine for 1,000 transactions but overwhelmed by a billion. Redshift is like hiring 1,000 specialized accountants (compute nodes), each handling a portion of the data simultaneously, and reporting results back to a head accountant (leader node) who combines everything into the final answer — in seconds.

OLTP vs OLAP — Key Distinction

Dimension	OLTP (e.g., RDS)	OLAP (e.g., Redshift)
Purpose	Day-to-day transactions (INSERT/UPDATE/DELETE)	Analytics & reporting (SELECT with aggregations)
Query Pattern	Simple, fast, high-volume	Complex, slow (seconds–minutes), low-volume
Data Volume	GBs to TBs	TBs to PBs
Storage	Row-oriented	Columnar
Example	Process a customer order	Total revenue by region for last 5 years

3.2 Core Concepts & Terminology

Term	Definition
Leader Node	Receives queries, builds query execution plan, coordinates Compute Nodes, returns results.
Compute Node	Executes the query plan in parallel. Each node has slices (parallel processing units).
Node Slice	A partition of a Compute Node's memory and disk. Each slice processes its data share independently.
Columnar Storage	Data stored column-by-column instead of row-by-row. Dramatically faster for aggregation queries (SUM, AVG, COUNT) because only relevant columns are read from disk.

Term	Definition
Distribution Style	How table rows are distributed across node slices: EVEN (round-robin), KEY (by column hash), ALL (full copy on each node).
Sort Key	Column(s) Redshift sorts data on disk. Queries filtering on sort key columns skip irrelevant blocks (Zone Maps).
Redshift Spectrum	Extends Redshift queries directly to data in Amazon S3 without loading it first.
Concurrency Scaling	Automatically adds temporary compute capacity during query bursts to maintain performance.
AQUA	Advanced Query Accelerator — hardware-accelerated cache between S3 and Compute Nodes (RA3 instances).
RA3 Instance	Redshift instance type with managed storage — separates compute and storage, auto-scales storage in S3.
Materialized View	Pre-computed result set stored and automatically refreshed. Accelerates repeated complex queries.
Data Sharing	Share live data across Redshift clusters without copying, even across accounts.

3.3 Architecture & Working

```
CLIENT (BI Tools: QuickSight, Tableau, PowerBI, JDBC/ODBC apps)
↓ SQL Query via JDBC/ODBC/Redshift API
LEADER NODE
• Parses SQL → Builds Optimized Query Plan
• Determines which Compute Nodes handle which data
• Sends C++ compiled code (not SQL) to Compute Nodes
↓ Parallel execution plan
COMPUTE NODES [Node 1] [Node 2] ... [Node N]
Each node: 2–16 slices running in parallel
• Read columnar data from local SSD (dc2) or managed S3 (ra3)
• Apply filters, aggregations, joins on their data partition
• Intermediate results sent back to Leader Node
↓ Aggregated result
LEADER NODE — merges partial results → final result → CLIENT
```

Columnar Storage Deep Dive

In a row-oriented database, to calculate SUM(revenue) you must read every column of every row. In Redshift's columnar storage, only the 'revenue' column is read — reducing I/O by up to 90%. Columnar storage also enables aggressive compression (same-type data in a column compresses better), further reducing storage costs and improving query speed.

Distribution Styles

Style	How It Works	Best For
EVEN	Rows distributed round-robin across all slices	Large tables without a natural join key

Style	How It Works	Best For
KEY	Rows with the same key value go to the same slice	Large fact tables joined with dimension tables on the key
ALL	Full copy of table on every Compute Node	Small dimension tables (eliminates data movement during joins)
AUTO	Redshift automatically chooses the best style	Default — let Redshift optimize for you

3.4 Major Features

Feature	Description	Benefit
Redshift Spectrum	Query data in S3 directly from Redshift using SQL	No ETL needed — query raw S3 data at scale
Concurrency Scaling	Auto-adds read clusters during peak query times	Consistent performance for hundreds of concurrent users
Automatic Table Optimization	Redshift analyzes query patterns and auto-adjusts sort/distribution keys	Self-tuning warehouse — less DBA effort
Materialized Views	Pre-compute expensive query results; auto-refresh on data change	Sub-second response for dashboards
Data Sharing	Share live Redshift data across clusters and accounts without copying	Real-time analytics across organizational boundaries
Redshift ML	Create ML models in SQL using Amazon SageMaker under the hood	Data analysts can do ML without leaving SQL
Serverless	Run Redshift workloads without provisioning clusters; auto-scales	Pay per use; zero infrastructure management
AQUA	Distributed hardware-accelerated cache layer for RA3 nodes	Up to 10x faster for S3-backed queries

3.5 Real-World Use Cases

Industry	Use Case
Retail — BI Reporting	Query 3 years of daily transaction history to produce weekly sales reports across 5,000 stores.
Fintech — Risk Analytics	Run Monte Carlo simulations on portfolio data for daily VaR (Value-at-Risk) reporting.
Healthcare — Claims Analysis	Analyze 10 years of insurance claims (50TB) to detect fraud patterns and cost drivers.
E-Commerce — Funnel Analysis	Track user journey from landing page to purchase across billions of clickstream events.
Gaming — Player Analytics	Report daily/monthly active users, churn rates, and in-app purchase trends.
Logistics — Supply Chain	Aggregate shipment data from 200 distribution centers to optimize routing and inventory.

3.6 Practical Example — Redshift Queries

```
-- Create a fact table with optimal keys
CREATE TABLE sales_fact (
sale_id BIGINT,
product_id INT,
store_id INT,
sale_date DATE,
amount DECIMAL(12,2)
)
DISTKEY(product_id) -- join key with product dimension
SORTKEY(sale_date); -- most queries filter by date

-- Query: Total revenue by category for 2024
SELECT p.category, SUM(s.amount) AS total_revenue
FROM sales_fact s
JOIN product_dim p ON s.product_id = p.product_id
WHERE s.sale_date BETWEEN '2024-01-01' AND '2024-12-31'
GROUP BY p.category
ORDER BY total_revenue DESC;

-- Redshift ML: predict churn using SageMaker under the hood
CREATE MODEL churn_model FROM customers_training
TARGET churn_flag
FUNCTION predict_churn
IAM_ROLE 'arn:aws:iam::123:role/RedshiftMLRole'
AUTO OFF SETTINGS (S3_BUCKET 'my-ml-bucket');
```

3.7 Integrations & Comparison

Service	Integration
Amazon S3	Redshift COPY loads bulk data; Spectrum queries S3 data in-place.
AWS Glue	Glue ETL loads transformed data into Redshift. Glue Catalog tables visible via Spectrum.
Amazon QuickSight	Native Redshift connector for BI dashboards with SPICE in-memory acceleration.
Amazon Kinesis Firehose	Streams real-time data directly into Redshift tables (via S3 staging).
Amazon EMR	EMR Spark writes to Redshift via JDBC or the Redshift Spark connector.
AWS DMS	Database Migration Service migrates on-premises data warehouses into Redshift.

Redshift vs Traditional DWH vs Athena

Dimension	Amazon Redshift	Athena	Teradata (On-Prem)
Type	Managed cloud DWH	Serverless SQL-on-S3	On-premises DWH appliance
Cost	~$0.25/node-hr (RA3)	$5/TB scanned	$100K–$10M+ hardware
Performance	Sub-second (with AQUA)	Seconds to minutes	Seconds (expensive tuning)

Dimension	Amazon Redshift	Athena	Teradata (On-Prem)
Management	Low (managed)	Zero	High (DBA team needed)
Persistent Storage	Yes (columnar, compressed)	S3 only	Yes (expensive disks)
Best For	Regular heavy analytics, BI	Ad-hoc queries on raw S3	Legacy large enterprises

Service 4

Amazon Kinesis

Real-Time Data Streaming — Ingest, Process, and Analyze Live Data

4.1 Simple Introduction

What is Amazon Kinesis?

Amazon Kinesis is a platform of services for collecting, processing, and analyzing real-time streaming data at any scale. Instead of waiting for batch jobs to run every hour or night, Kinesis lets you react to data as it arrives — in milliseconds to seconds. Kinesis is a family of four services, each addressing a different streaming need.

Real-World Analogy

■ **Kinesis is like a river system.** Data flows continuously like water in a river. Kinesis Data Streams is the main river channel — data producers (sensors, apps) pour data in, and consumers (analytics, Lambda) scoop data out. Kinesis Firehose is an automated pump that moves water to a reservoir (S3, Redshift). Kinesis Data Analytics is a water treatment plant that processes and transforms water as it flows through. You cannot pause the river, but Kinesis ensures no drop is lost.

The Four Kinesis Services

Service	Purpose	Key Feature
Kinesis Data Streams (KDS)	Durable, ordered stream of records. Multiple consumers can read in parallel.	Custom, real-time consumer apps
Kinesis Data Firehose	Fully managed delivery service — automatically delivers to S3, Redshift, OpenSearch, Splunk.	Zero-code streaming ETL to destinations
Kinesis Data Analytics	Run Apache Flink applications (or legacy SQL) on streaming data.	Real-time stream processing with Flink
Kinesis Video Streams	Ingest, store, and analyze live video streams from cameras and devices.	ML on live video; playback

4.2 Core Concepts — Kinesis Data Streams

Concept	Definition
Stream	A named, ordered sequence of data records. Records are organized into Shards.
Shard	The fundamental throughput unit of a Stream. Each shard provides: 1 MB/s ingestion, 2 MB/s consumption, 1,000 records/s PUT.
Record	The unit of data in a stream. Each record has a Partition Key, Sequence Number, and Data Blob (up to 1 MB).
Partition Key	A user-defined string that determines which shard a record is routed to (via MD5 hash). Same key = same shard = ordered delivery.
Sequence Number	Unique identifier assigned by Kinesis to each record in its shard. Monotonically increasing.

Concept	Definition
Retention Period	How long records stay in the stream: default 24 hours, configurable up to 365 days.
Producer	Application that writes data to the stream (using SDK, KPL — Kinesis Producer Library, or Agent).
Consumer	Application that reads data from the stream (using KCL — Kinesis Consumer Library, Lambda, Firehose, or Analytics).
Enhanced Fan-Out	Each consumer gets a dedicated 2 MB/s throughput per shard (instead of shared 2 MB/s). Low latency (~70ms).
On-Demand Mode	Kinesis automatically manages shard count — scales up/down based on traffic. No manual shard management.

4.3 Architecture & Working

```
PRODUCERS
Web Servers ■ IoT Sensors ■ Mobile Apps ■ Click Streams ■ Log Agents
↓ PutRecord / PutRecords API (Partition Key assigned)
KINESIS DATA STREAMS
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Shard 1: [Rec1][Rec2][Rec3]... (Key hash: 0-33%) ■
■ Shard 2: [Rec4][Rec5][Rec6]... (Key hash: 33-66%) ■
■ Shard 3: [Rec7][Rec8][Rec9]... (Key hash: 66-100%) ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
Records retained for 24h–365 days (multiple consumers re-read)
↓ GetRecords / SubscribeToShard (Enhanced Fan-Out)
CONSUMERS
AWS Lambda ■ KCL App (EC2/ECS) ■ Firehose ■ Kinesis Analytics
```

Kinesis Data Firehose Architecture

```
SOURCES: Direct PUT ■ KDS ■ MSK ■ CloudWatch Logs ■ WAF
↓
FIREHOSE DELIVERY STREAM
Optional: Lambda transformation (filter, enrich, convert format)
Optional: Format conversion (JSON → Parquet/ORC via Glue schema)
Buffering: by size (1–128 MB) or time (60–900 seconds)
↓
DESTINATIONS: S3 ■ Redshift (via S3 staging) ■ OpenSearch ■ Splunk ■ HTTP endpoint
```

4.4 Features

Feature	Service	Description
Server-Side Encryption	KDS	Encrypts stream data at rest using KMS keys.

Feature	Service	Description
Enhanced Fan-Out	KDS	2 MB/s per consumer per shard with HTTP/2 push — low latency reads.
On-Demand Capacity	KDS	Auto-scale shards without manual intervention.
Lambda Integration	KDS	Trigger Lambda functions for each batch of records — serverless stream processing.
Dynamic Partitioning	Firehose	Route records to different S3 prefixes/paths based on record content (e.g., by region or customer_id).
Format Conversion	Firehose	Auto-convert JSON to Parquet or ORC using Glue schema for Athena optimization.
Apache Flink Runtime	Analytics	Run stateful stream processing applications using Apache Flink APIs.
Auto Scaling (Analytics)	Analytics	Automatically scales Flink application parallelism based on incoming data rate.
Video Playback & Fragments	Video Streams	Store video as fragments; playback via HLS or DASH. Integrate with Rekognition for real-time analysis.

4.5 Real-World Use Cases

Industry	Use Case	Kinesis Service
E-Commerce	Real-time fraud detection on each transaction as it occurs	KDS + Lambda
IoT / Manufacturing	Ingest sensor data from 100,000 factory machines; detect anomalies instantly	KDS + Analytics (Flink)
Gaming	Track player events (kills, deaths, purchases) in real time for live leaderboards	KDS + DynamoDB
Fintech	Stream stock trade data to analytics engine; trigger alerts on price spikes	KDS + Analytics (Flink)
Media Streaming	Deliver live video from news cameras to broadcast platform with ML-based content moderation	Video Streams + Rekognition
Log Analytics	Stream application logs from 1,000 servers into OpenSearch for real-time monitoring	Firehose → OpenSearch
Clickstream Analytics	Capture every user click on e-commerce site; load to Redshift for funnel analysis	Firehose → Redshift

4.6 Practical Example — Clickstream to S3 with Firehose

```
import boto3, json, time

firehose = boto3.client('firehose', region_name='us-east-1')

def send_click_event(user_id, page, action):
record = {
```

```
'user_id': user_id,
'page': page,
'action': action,
'timestamp': int(time.time())
}
firehose.put_record(
DeliveryStreamName='clickstream-to-s3',
Record={'Data': json.dumps(record) + '\n'}
)

# Simulate 1000 user clicks
for i in range(1000):
send_click_event(f'user_{i}', '/product/456', 'add_to_cart')
```

■ *Note: Firehose buffers records and delivers them to S3 every 60 seconds or every 5 MB — whichever comes first. Data auto-partitioned by date prefix.*

4.7 Comparison: KDS vs Firehose vs MSK

Dimension	Kinesis Data Streams	Kinesis Firehose	Amazon MSK (Kafka)
Management	Semi-managed (shard management)	Fully managed, serverless	Managed Kafka (still complex)
Latency	~200ms	60s–15min (buffering)	~10ms
Consumer Code	Required (KCL/Lambda)	None — fully automated	Required (Kafka consumers)
Destinations	Custom — any	Fixed set (S3, Redshift, etc.)	Custom — any
Replay	Yes (up to 365 days)	No replay	Yes (Kafka retention)
Protocol	Kinesis API	Kinesis API	Kafka protocol
Best For	Custom real-time consumers	Load streaming data to S3/Redshift	Kafka-compatible ecosystems

Service 5

AWS Glue

Serverless ETL Service & Unified Data Catalog

5.1 Simple Introduction

What is AWS Glue?

AWS Glue is a fully managed, serverless Extract, Transform, and Load (ETL) service that makes it easy to discover, prepare, and combine data for analytics, ML, and application development. Glue automatically discovers the structure of your data (schema), generates ETL code, and runs it on a managed Apache Spark environment — all without provisioning any servers. At its core sits the **AWS Glue Data Catalog** — a central metadata repository that acts as the single source of truth for all your data assets.

Real-World Analogy

■ **Glue is like a universal translator and delivery service for your data.** Imagine you have data in 10 different languages (CSV, JSON, Parquet, Avro) from 10 different countries (S3, RDS, DynamoDB, Redshift). AWS Glue automatically reads all languages, translates them into a common format, and delivers them to wherever they're needed — without you having to write a translation program for each pair.

ETL Defined

Phase	What Happens	Example
Extract (E)	Read data from one or more sources	Read CSV from S3, read rows from RDS MySQL
Transform (T)	Clean, filter, join, aggregate, convert	Remove nulls, join customer + orders, convert to Parquet
Load (L)	Write transformed data to destination	Write to S3, load into Redshift, insert into DynamoDB

5.2 Core Concepts & Components

Component	Description
Glue Data Catalog	Central metadata store — databases, tables, schemas, partitions. Used by Athena, EMR, Redshift Spectrum.
Glue Crawler	Automatically scans data sources (S3, RDS, DynamoDB) and populates the Data Catalog with schema/metadata.
Glue ETL Job	A Spark-based (or Python shell) script that reads, transforms, and writes data. Auto-generated or custom.
Glue Studio	Visual drag-and-drop interface to build ETL pipelines without writing code.
Glue DataBrew	Visual data profiling and cleaning tool — 250+ pre-built transformations, no code required.
Glue Streaming ETL	Run Glue ETL jobs continuously against streaming sources (Kinesis, Kafka) instead of batch.

Component	Description
DPU (Data Processing Unit)	Unit of Glue compute: 4 vCPU + 16 GB RAM. You pay per DPU-hour. Min 2 DPUs for standard jobs.
Glue Schema Registry	Centralized schema versioning and validation for Kafka, Kinesis, and MSK data streams.
Connection	Credentials and network config for Glue to access JDBC sources (RDS, Redshift) in VPC.
Trigger	Schedule or event that starts Glue jobs: on-demand, time-based (cron), or event-based (job completion).
DynamicFrame	Glue's extension of Spark DataFrame — handles schema inconsistencies and nested types gracefully.

5.3 Architecture & Working

```
DATA SOURCES
Amazon S3 ■ Amazon RDS/Aurora ■ Amazon Redshift ■ DynamoDB ■ JDBC (on-prem DB)
↓ [Glue Crawlers scan → infer schema]
GLUE DATA CATALOG (Hive Metastore compatible)
Databases → Tables → Partitions → Schemas → Statistics
↓ [Shared by all analytics services]
Athena ■ Redshift Spectrum ■ EMR ■ Lake Formation
↓ [Glue ETL Job triggered]
GLUE SPARK ENVIRONMENT (serverless, managed)
Reads source → Applies transformations (DynamicFrames) → Writes output
[Glue Studio visual pipeline OR custom PySpark/Scala script]
↓
DESTINATIONS: S3 (Parquet) ■ Redshift ■ RDS ■ OpenSearch ■ DynamoDB
```

Glue Crawler Workflow

- **1.** You configure a Crawler pointing to S3 paths, RDS tables, or DynamoDB.
- **2.** Crawler samples data, infers schema (column names, types, partitions).
- **3.** Crawler writes table definitions into the Glue Data Catalog.
- **4.** Athena, EMR, and Redshift Spectrum can now query the data immediately using the catalog.
- **5.** Crawlers can run on a schedule — new data added to S3 triggers re-crawling to update partitions.

5.4 Features

Feature	Description	Use Case
Auto Code Generation	Glue generates PySpark/Scala ETL code from your source/target mapping	Speed up ETL development — edit generated code as needed
Job Bookmarks	Tracks which data has already been processed — incremental loads only	Avoid reprocessing old data on each run

Feature	Description	Use Case
Glue Studio Visual ETL	Drag-and-drop pipeline builder with 50+ built-in transforms	Non-engineers can build ETL workflows
DataBrew	250+ one-click data cleaning transformations with visual profiling	Data analysts clean messy data without code
Streaming ETL	Process Kinesis or Kafka streams continuously in near-real-time	Real-time data pipeline without managing Flink/Spark clusters
Schema Registry	Validate, version, and evolve Kafka/Kinesis message schemas	Prevent schema drift breaking downstream consumers
Flex Execution	Run non-urgent Glue jobs on spare AWS capacity at up to 34% lower cost	Cost optimization for overnight batch jobs

5.5 Real-World Use Cases

Scenario	Details
Daily Sales ETL	Nightly: Crawl new CSV files in S3 → Glue job transforms to Parquet → loads to Redshift for BI.
Data Lake Cataloging	Crawl 500 S3 prefixes across 20 departments → unified Data Catalog → Athena queries without ETL.
Database Migration ETL	Migrate 200 Oracle tables to S3 Parquet. Glue generates ETL scripts for each table automatically.
Real-Time Log Processing	Glue Streaming ETL reads CloudWatch Logs from Kinesis → enriches with Glue DynamicFrames → writes to OpenSearch.
Data Quality Pipeline	DataBrew profiles incoming marketing data → flags anomalies → cleaned data loaded to Redshift.

5.6 Practical Example — PySpark ETL Job

```
import sys
from awsglue.transforms import *
from awsglue.utils import getResolvedOptions
from pyspark.context import SparkContext
from awsglue.context import GlueContext
from awsglue.job import Job

sc = SparkContext()
glueContext = GlueContext(sc)
job = Job(glueContext)

# Read from Glue Data Catalog (source table)
datasource = glueContext.create_dynamic_frame.from_catalog(
database='sales_db',
table_name='raw_orders'
)

# Transform: drop nulls, rename columns
transformed = datasource.drop_nulls()
transformed = RenameField.apply(transformed, 'ord_id', 'order_id')
```

```
# Write to S3 as Parquet (partitioned by year/month)
glueContext.write_dynamic_frame.from_options(
frame=transformed,
connection_type='s3',
connection_options={'path': 's3://curated-bucket/orders/', 'partitionKeys':
['year','month']},
format='parquet'
)
job.commit()
```

5.7 Integrations & Comparison

Service	Integration with Glue
Amazon S3	Primary source and destination for Glue ETL jobs. Glue Crawlers scan S3 paths.
Amazon Athena	Athena uses Glue Data Catalog for all table definitions — zero additional config needed.
Amazon Redshift	Glue loads data into Redshift via JDBC. Redshift Spectrum uses Glue Catalog.
Amazon EMR	EMR uses Glue Data Catalog as its Hive Metastore (shared schema layer).
AWS Lake Formation	Lake Formation governs access to Glue Catalog tables with fine-grained permissions.
Amazon Kinesis	Glue Streaming ETL consumes records from Kinesis Data Streams continuously.
Amazon MSK	Glue Streaming ETL can also read from MSK Kafka topics.

Glue vs EMR vs Athena for ETL

Criterion	AWS Glue	Amazon EMR	Amazon Athena
ETL Code	Auto-generated PySpark + custom	Custom Spark/Hive — full control	CTAS (Create Table As Select)
Management	Serverless — zero infra	Managed cluster — some ops	Serverless — zero infra
Cost Model	Per DPU-hour	Per EC2 instance-hour	Per TB scanned
Best For	Standard ETL, cataloging, DataBrew	Complex ML, custom streaming, large-scale batch	Ad-hoc ETL via SQL; simpler transformations
Learning Curve	Low–Medium	Medium–High	Low (SQL)

Service 6

Amazon MSK

Fully Managed Apache Kafka — Enterprise-Grade Event Streaming

6.1 Simple Introduction

What is Amazon MSK?

Amazon Managed Streaming for Apache Kafka (MSK) is a fully managed service that makes it easy to build and run applications that use Apache Kafka for processing streaming data. Apache Kafka is an open-source distributed event streaming platform used by thousands of companies for high-throughput, low-latency data pipelines, real-time analytics, and event-driven architectures. MSK provisions, configures, and maintains Kafka clusters on your behalf, handling broker nodes, ZooKeeper/KRaft nodes, and storage automatically.

What is Apache Kafka? (Key Concepts)

Kafka is a distributed commit log: producers write events (messages) to named channels called **topics**. Topics are split into **partitions** for parallelism. Messages are durably replicated across multiple **broker** nodes. Consumer groups read from partitions independently at their own pace, enabling multiple applications to consume the same event stream.

Real-World Analogy

■ **Kafka/MSK is like a newspaper publishing system.** The newspaper (topic) publishes new editions continuously. Reporters (producers) contribute articles. Different departments — advertising, distribution, digital — all receive the same newspaper and process it independently at their own speed. Importantly, the newspaper archive (Kafka retention) keeps all back issues — any department can re-read yesterday's paper anytime. MSK is like renting the entire printing press infrastructure without having to manage the machinery.

6.2 Core Concepts & Terminology

Term	Definition
Broker	A Kafka server that stores and serves messages. MSK runs 2–N brokers across AZs for HA.
Topic	A named feed to which records are published. Like a database table in a streaming world.
Partition	Topics are divided into partitions for parallelism. Each partition is an ordered, immutable log.
Replication Factor	Number of broker copies for each partition. RF=3 means 3 copies across 3 brokers — survives 2 broker failures.
Producer	Application that publishes (writes) records to a Kafka topic.
Consumer Group	A group of consumers sharing the work of reading a topic. Each partition assigned to one consumer in the group.

Term	Definition
Offset	A numeric position within a partition. Consumers track their offset — enabling replay from any point.
ZooKeeper	(Legacy) Coordination service for Kafka cluster metadata. Being replaced by KRaft (Kafka Raft).
KRaft Mode	Kafka's built-in consensus protocol replacing ZooKeeper. MSK supports KRaft clusters.
MSK Connect	Managed Apache Kafka Connect — run connectors to integrate Kafka with external systems (S3, databases).
MSK Serverless	Run Kafka without provisioning broker nodes — auto-scales, pay per usage.

6.3 Architecture & Working

```
PRODUCERS (Microservices ■ Mobile Apps ■ IoT Devices ■ Databases via CDC)
↓ kafka-clients library (batch, compression, retry logic)
MSK CLUSTER
■■■■■■■■■■■■■■■■■ Multi-AZ Deployment ■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Broker 1 (AZ-a) ■ Broker 2 (AZ-b) ■ Broker 3 (AZ-c) ■
■ Topic: orders — Partition 0 (Leader on B1, Replicas on B2,B3)■
■ Topic: orders — Partition 1 (Leader on B2, Replicas on B1,B3)■
■ Topic: orders — Partition 2 (Leader on B3, Replicas on B1,B2)■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
↓ Consumer Group reads (each partition → one consumer)
CONSUMER GROUPS
Group A (Fraud Service): Reads all partitions → real-time fraud checks
Group B (Analytics): Reads all partitions → writes to S3 via MSK Connect
Group C (Notifications): Reads all partitions → sends email/SMS alerts
```

MSK vs Self-Managed Kafka

Task	Self-Managed	Amazon MSK
Broker provisioning	Manual EC2 setup + Kafka install	1-click cluster creation
ZooKeeper management	Manual ZK ensemble setup	Fully managed by AWS
Broker upgrades	Manual rolling upgrade (risky)	Managed upgrades with minimal downtime
Monitoring	Custom Prometheus/Grafana setup	CloudWatch + Open Monitoring (Prometheus)
Security	Manual TLS/SASL/ACL config	Encryption + IAM auth + TLS built-in
Scaling	Manual partition rebalancing	MSK Serverless auto-scales; managed resizing

6.4 Features

Feature	Description
Multi-AZ Replication	Brokers deployed across 3 AZs automatically — survives AZ failures without data loss.
MSK Connect	Fully managed Kafka Connect — 100+ pre-built connectors for S3, Redshift, Salesforce, MongoDB.
MSK Serverless	Run Kafka workloads without capacity planning — auto-scales based on throughput.
Open Monitoring	Expose Kafka metrics to Prometheus for custom Grafana dashboards.
IAM Access Control	Native IAM authentication for Kafka clients — no SASL/plaintext credentials.
Encryption	TLS in-transit between all Kafka clients and brokers; at-rest encryption via KMS.
Tiered Storage	Store older Kafka messages in S3 at low cost (configurable retention tiers).
Schema Registry Integration	Integrate with Glue Schema Registry to validate message schemas before ingestion.

6.5 Real-World Use Cases

Industry	Use Case
Fintech	Event sourcing — every financial event (trade, payment, transfer) written to Kafka as immutable event log.
E-Commerce	Order pipeline: order-placed → inventory-check → payment-processed → shipment-created (event-driven saga).
Telecom	CDR (Call Detail Record) streaming — billions of call records processed in real time for billing and fraud.
Healthcare	HL7 FHIR events streamed from hospital systems to analytics platform for patient monitoring.
Ride-Sharing	Driver location events (millions/sec) streamed to matching engine and route optimization.
Log Aggregation	Centralize application logs from 10,000 microservices via Kafka into OpenSearch for observability.

6.6 Practical Example — Python Producer & Consumer

```
# Install: pip install kafka-python
from kafka import KafkaProducer, KafkaConsumer
import json

# PRODUCER — send order events to MSK
producer = KafkaProducer(
bootstrap_servers=['b-1.msk-cluster.xxx.kafka.us-east-1.amazonaws.com:9092'],
value_serializer=lambda v: json.dumps(v).encode('utf-8')
)
producer.send('orders', {'order_id': 'ORD-001', 'amount': 129.99, 'status': 'placed'})
producer.flush()

# CONSUMER — fraud detection service reads orders
```

```
consumer = KafkaConsumer(
'orders',
bootstrap_servers=['b-1.msk-cluster.xxx.kafka.us-east-1.amazonaws.com:9092'],
group_id='fraud-detection-service',
value_deserializer=lambda m: json.loads(m.decode('utf-8'))
)
for msg in consumer:
order = msg.value
if order['amount'] > 10000:
print(f'HIGH-VALUE ALERT: {order}')
```

6.7 Comparison: MSK vs Kinesis Data Streams

Dimension	Amazon MSK (Kafka)	Kinesis Data Streams
Protocol	Apache Kafka protocol (open standard)	Proprietary Kinesis API
Ecosystem	Kafka ecosystem: Connect, Streams, ksqlDB, Flink	AWS ecosystem: Lambda, Firehose, Analytics
Partitions	Unlimited (you set partition count)	Up to 10,000 shards (auto-scaling available)
Retention	Configurable (hours to unlimited)	24h default; up to 365 days
Consumer Groups	Multiple independent consumer groups	Multiple consumers with Enhanced Fan-Out
Latency	~10ms	~200ms
Management	More operational overhead	Less — more managed
Ordering	Per partition (strong ordering)	Per shard (strong ordering)
Best For	Existing Kafka users; event sourcing; CDC	AWS-native real-time pipelines; simpler setup

Service 7

Amazon OpenSearch Service

Search, Log Analytics & Real-Time Monitoring at Scale

7.1 Simple Introduction

What is Amazon OpenSearch Service?

Amazon OpenSearch Service (formerly Amazon Elasticsearch Service) is a fully managed service for deploying, operating, and scaling OpenSearch and legacy Elasticsearch clusters in AWS. OpenSearch is an open-source search and analytics engine derived from Elasticsearch. It enables you to perform full-text search, log analysis, real-time application monitoring, and clickstream analytics. OpenSearch Dashboards (formerly Kibana) provides a powerful visualization interface for exploring and visualizing your data.

Real-World Analogy

■ **OpenSearch is like Google Search for your own data.** When you type in Google, it instantly searches billions of web pages. OpenSearch does the same thing but for YOUR data — application logs, product catalogs, documents, user records. It uses inverted indexes (like a book index mapping words to page numbers) to find documents matching your search in milliseconds, even across billions of records.

7.2 Core Concepts & Terminology

Term	Definition
Index	A collection of documents with similar characteristics — like a database table. Stored in shards.
Document	The basic unit of information in OpenSearch — a JSON object (like a database row).
Shard	A subset of an index. OpenSearch distributes shards across nodes for parallelism and fault tolerance.
Replica Shard	A copy of a primary shard on a different node. Provides high availability and read scalability.
Mapping	Schema definition for an index — defines field names, types, and analyzers.
Analyzer	Pipeline that converts text to searchable tokens: tokenizer + token filters (lowercase, stemming, stop words).
Inverted Index	Core data structure: maps terms (words) to the list of documents containing them. Enables sub-millisecond full-text search.
Query DSL	OpenSearch Query Domain Specific Language — JSON-based API for building complex search queries.
OpenSearch Dashboards	Web UI for creating charts, maps, and dashboards on OpenSearch data (like Kibana).
Domain	AWS term for an OpenSearch cluster — includes nodes, storage, and access control settings.

Term	Definition
UltraWarm	Low-cost warm storage tier using S3-backed nodes — for less-frequently accessed log data.
Cold Storage	Even cheaper archival tier for logs you rarely query — data moved to and from on demand.

7.3 Architecture & Working

```
DATA SOURCES
App Logs (Logstash/Fluentd) ■ Kinesis Firehose ■ Lambda ■ CloudWatch ■ Beats
↓ Index API (HTTP POST /index/_doc)
OPENSEARCH DOMAIN
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■ Coordinating Node: Routes requests to correct shards ■
■ Data Node 1 (AZ-a): Shard 0 Primary ■ Shard 1 Replica ■
■ Data Node 2 (AZ-b): Shard 1 Primary ■ Shard 0 Replica ■
■ Data Node 3 (AZ-c): Shard 2 Primary ■ Shard 1 Replica ■
■ UltraWarm Nodes: [Older data on S3-backed EBS] ■
■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
↓ Search API (GET /index/_search)
CONSUMERS
OpenSearch Dashboards (Kibana) ■ Custom App ■ Grafana ■ Alerting Engine
```

How a Search Request Works

- **1. Query Phase:** Coordinating node broadcasts the query to all relevant shards (primary or replica).
- **2. Local Search:** Each shard searches its local inverted index, returns top-N matching document IDs with relevance scores.
- **3. Reduce Phase:** Coordinating node merges all shard results, sorts by score, returns top-N overall.
- **4. Fetch Phase:** Coordinating node fetches full document source from the shards holding the top-N documents.
- **5. Response:** JSON response with hits, total count, and relevance scores returned in milliseconds.

7.4 Features

Feature	Description
Full-Text Search	Analyze and search free-text fields with relevance scoring, fuzzy matching, and synonyms.
OpenSearch Dashboards	Visual analytics: time-series charts, geo-maps, histograms, metric panels — all on top of OpenSearch.
UltraWarm & Cold Storage	Two-tier cost optimization: move older logs from hot SSD nodes to warm (S3-backed) and cold storage.
Alerting	Define monitors and triggers — get notified via SNS, Slack, PagerDuty when anomalies occur.

Feature	Description
Anomaly Detection	ML-powered automatic detection of anomalies in time-series data without writing rules.
Index State Management (ISM)	Automatically transition indices through lifecycle stages (hot → warm → cold → delete) on a schedule.
k-NN (k-Nearest Neighbor)	Vector similarity search — find semantically similar documents for AI/ML embedding use cases.
SQL Plugin	Query OpenSearch using SQL syntax instead of Query DSL — lower learning curve.
Fine-Grained Access Control	Field-level and document-level security — restrict which users can see which data.

7.5 Real-World Use Cases

Scenario	Details
Log Analytics (ELK Stack)	Ingest logs from 1,000 microservices via Fluentd → OpenSearch → Dashboards for real-time monitoring and alerting.
E-Commerce Search	Product catalog search: full-text search, faceted filtering (category, price range, rating), autocomplete.
Security Analytics (SIEM)	Ingest CloudTrail, VPC Flow Logs, GuardDuty findings → detect attack patterns using Anomaly Detection.
Application Performance Monitoring	Track request latency, error rates, throughput across microservices in real time.
AI-Powered Semantic Search	Store text embeddings from ML models → k-NN search to find similar documents semantically.
Healthcare — Clinical Search	Search medical records, clinical notes, and lab results with structured and full-text queries.

7.6 Practical Example — Index & Search

```
import boto3, requests, json

OPENSEARCH_ENDPOINT = 'https://my-domain.us-east-1.es.amazonaws.com'

# Index a product document
doc = {
'product_name': 'Wireless Bluetooth Headphones',
'category': 'Electronics',
'price': 79.99,
'description': 'Premium sound quality with 30hr battery life'
}
r = requests.post(f'{OPENSEARCH_ENDPOINT}/products/_doc/1', json=doc)

# Full-text search with filter
query = {
'query': {
'bool': {
'must': [{'match': {'description': 'battery life'}}],
```

```
'filter': [{'range': {'price': {'lte': 100}}}]
}
}
}
results = requests.get(f'{OPENSEARCH_ENDPOINT}/products/_search', json=query)
print(json.dumps(results.json()['hits']['hits'], indent=2))
```

7.7 Comparison: OpenSearch vs CloudWatch vs Athena for Log Analytics

Dimension	OpenSearch	CloudWatch Logs Insights	Athena on S3
Search Type	Full-text + structured queries	Structured query language (CloudWatch QL)	SQL on structured/semi-structured logs
Real-Time	Yes — near real-time	Yes	No — batch query
Visualization	OpenSearch Dashboards (rich)	CloudWatch Dashboards (basic)	None native (need QuickSight)
Anomaly Detection	Built-in ML	Built-in (simpler)	None
Cost	Per node-hour (can be expensive)	Per GB ingested + queried	Per TB scanned
Best For	Complex search + rich dashboards + ML	AWS-native log analytics (simpler)	Ad-hoc log analysis on S3 at low cost

Service 8

Amazon QuickSight

Serverless Business Intelligence — Dashboards, ML Insights & Embedded Analytics

8.1 Simple Introduction

What is Amazon QuickSight?

Amazon QuickSight is a fully managed, serverless cloud Business Intelligence (BI) and data visualization service. It enables business users, analysts, and data scientists to create interactive dashboards, charts, and reports from virtually any data source — all without managing servers or BI infrastructure. QuickSight can scale to serve thousands of users simultaneously, and its pay-per-session pricing model makes it significantly more cost-effective than traditional BI tools like Tableau or Power BI for large user bases.

Real-World Analogy

■ **QuickSight is like a smart TV dashboard for your business data.** Imagine your entire company's data — sales, marketing, operations — displayed on an executive dashboard like a smart TV. Everyone from the CEO to a regional manager can see their relevant numbers in beautiful charts, updated in real time. QuickSight is that smart TV dashboard, but connected to all your AWS data sources and smart enough to tell you 'Hey, sales in Region X dropped 20% this week — here's why.'

8.2 Core Concepts & Terminology

Term	Definition
SPICE	Super-fast Parallel In-memory Calculation Engine — QuickSight's in-memory data store that delivers sub-second query performance at any scale.
Dataset	Data imported or connected to QuickSight from a source. Can be stored in SPICE or queried directly.
Analysis	The workspace where you create and edit visualizations. Contains multiple visuals and filters.
Dashboard	A read-only, published version of an Analysis. Shareable with specific users/groups.
Visual	A single chart or table within an Analysis (bar chart, line graph, pie chart, heat map, etc.).
Calculated Field	Custom metric or dimension derived from dataset fields using QuickSight functions (IF, SUM, DATEDIFF, etc.).
ML Insights	Built-in ML features: Anomaly Detection, Forecasting, and Auto-Narratives generated automatically.
Embedded Analytics	Embed QuickSight dashboards directly into your own web application using the Embedding SDK.
Row-Level Security (RLS)	Restrict which data rows a user sees based on their identity — one dashboard, personalized per user.
Q (QuickSight Q)	Natural Language Query feature — users type plain English questions ('Show me revenue by region this year') to get instant charts.

Term	Definition
Paginated Reports	Pixel-perfect reports for printing or PDF export — for operational reporting needs.

8.3 Architecture & Working

```
DATA SOURCES
Amazon S3 ■ Redshift ■ Athena ■ RDS/Aurora ■ Snowflake ■ Salesforce ■ Excel
↓ [Import into SPICE or Direct Query mode]
SPICE ENGINE (In-Memory, Auto-Scaled by AWS)
Data stored in columnar format → sub-second queries for millions of rows
OR
DIRECT QUERY MODE
Every chart click sends SQL to Redshift/Athena/RDS — always current data
↓ [Analysis workspace]
QUICKSIGHT AUTHOR: Creates Analyses → Publishes Dashboards
Adds ML Insights: Anomaly Detection ■ Forecasting ■ Auto-Narratives
↓ [Share or Embed]
CONSUMERS
Dashboard Readers (pay-per-session) ■ Embedded in Custom Web Apps ■ PDF/Email Reports
```

SPICE vs Direct Query

Aspect	SPICE Mode	Direct Query Mode
Performance	Sub-second (in-memory)	Depends on source (seconds+)
Data Freshness	Refreshed on schedule (hourly/daily)	Always current
Cost	Included SPICE capacity; extra at $0.25/GB	Pay per query at source (Athena per TB)
Best For	Dashboards used by many readers; aggregate data	Real-time operational dashboards; very large datasets

8.4 Features

Feature	Description
30+ Visual Types	Bar, line, scatter, heat map, tree map, funnel, waterfall, geo map, KPI — all with drill-down.
ML-Powered Anomaly Detection	Continuously scans all measures — alerts you to statistically significant changes automatically.
Forecasting	Built-in time-series forecasting using ML — predict future trends with confidence intervals.
Auto-Narratives	Automatically generates plain English descriptions of what's happening in your charts.
QuickSight Q	NLQ (Natural Language Query) — type 'total sales in California last quarter' → instant chart.

Feature	Description
Embedded Analytics	Embed dashboards in SaaS apps, portals, or websites via Embedding SDK + anonymous/authenticated modes.
Paginated Reports	Highly formatted multi-page PDF/Excel reports for compliance, invoices, and operational reports.
Row-Level Security	Each user sees only their slice of data. One dashboard serves all users with personalized data.
Cross-Dataset Joins	Join multiple datasets visually without writing SQL.
Mobile App	Native iOS and Android app for viewing dashboards on mobile.

8.5 Real-World Use Cases

Industry	Use Case
Retail — Executive Dashboard	CEO sees daily revenue, regional performance, inventory alerts in a single dashboard updated every 15 minutes.
SaaS — Embedded Analytics	B2B SaaS company embeds QuickSight dashboards in their product so customers see their own usage analytics.
Healthcare — Operations	Hospital monitors bed occupancy, patient wait times, and staff utilization in real-time via QuickSight.
Fintech — Risk Dashboard	Risk managers see portfolio exposure, VaR breaches, and anomaly alerts — with drill-down to individual trades.
Marketing — Campaign Analytics	Marketing team tracks ad spend, conversion rates, and CPA across channels with QuickSight Q for ad-hoc questions.
Manufacturing — OEE Dashboard	Factory managers see Overall Equipment Effectiveness, downtime reasons, and shift performance.

8.6 Practical Example — Athena to QuickSight Dashboard

Step-by-step setup for a sales analytics dashboard:

- **Step 1:** In QuickSight → New Dataset → Athena → Select Glue catalog database 'sales_db' → table 'transactions'.
- **Step 2:** Choose SPICE import → click Import. QuickSight fetches 5M rows in ~30 seconds.
- **Step 3:** In Analysis → Add Visual → Bar Chart. X-axis: product_category, Y-axis: SUM(amount).
- **Step 4:** Add a KPI visual: Total Revenue. Add a Line Chart: Revenue over time (sale_date).
- **Step 5:** Add ML Insight → Anomaly Detection on Revenue — QuickSight automatically flags unusual days.
- **Step 6:** Publish as Dashboard → Share with Finance group → enable email subscription for weekly PDF report.
- **Step 7 (Embedding):** Generate an embed URL via the QuickSight API and inject into your web app's iframe.

8.7 Comparison: QuickSight vs Tableau vs PowerBI

Dimension	Amazon QuickSight	Tableau	Power BI
Pricing Model	$18–$54/author/mo + $0.30/session (readers)	$70–$115/user/month	$10–$20/user/month (Pro)
Serverless	Yes — fully managed	No — server required (Tableau Server)	No (Power BI Gateway for on-prem)
AWS Integration	Native (Redshift, Athena, S3, etc.)	Via connectors	Via connectors
ML Insights	Built-in (Anomaly, Forecast, NLQ)	Partial (Einstein Analytics)	Built-in (AI Insights)
Embedding	Yes (SDK, anonymous)	Yes (expensive)	Yes (Power BI Embedded)
Learning Curve	Low	High	Medium
Large User Scale	Excellent (pay-per-session)	Expensive at scale	More affordable

Service 9

Amazon Athena

Serverless Interactive SQL — Query S3 Data Without Moving It

9.1 Simple Introduction

What is Amazon Athena?

Amazon Athena is a serverless, interactive query service that lets you analyze data stored in Amazon S3 using standard SQL. You don't need to set up any infrastructure, manage clusters, or load data into a database. Simply point Athena at your S3 data, define a schema (or let Glue Crawlers define it for you), and start querying. Athena is built on **Presto** (now Trino), a high-performance distributed SQL query engine. You pay only **$5 per TB of data scanned** — making it extremely cost-effective for ad-hoc analytics.

Real-World Analogy

■■ **Athena is like Google Docs search — but for your data files.** Imagine you have millions of spreadsheets (CSV/Parquet files) in a giant file cabinet (S3). Normally, to answer 'What were total sales in Q3?' you'd have to load all files into a database first (hours of work). Athena is like a magic search feature that reaches directly into the file cabinet, reads only the relevant files, and gives you the SQL answer in seconds — without moving or copying anything.

Schema-on-Read vs Schema-on-Write

Approach	Definition	Example
Schema-on-Write (RDBMS)	Define schema BEFORE loading data. Data must conform to schema at load time.	CREATE TABLE → INSERT rows → query
Schema-on-Read (Athena)	Define schema WHEN querying. Raw files in S3 stay as-is.	Files already in S3 → CREATE EXTERNAL TABLE → instant query

9.2 Core Concepts & Terminology

Term	Definition
External Table	Table definition in the Glue Data Catalog that maps SQL columns to S3 file paths. No data is copied.
Data Catalog	AWS Glue Data Catalog — stores all table metadata used by Athena for query planning.
Workgroup	Logical grouping of Athena queries — set cost controls, S3 output location, encryption per workgroup.
Query Result Location	S3 location where Athena stores query results and metadata for each run.
Partition	Dividing S3 data by a column (e.g., year/month/day) so Athena reads only relevant folders.
Partition Projection	Configure Athena to auto-generate partitions from the S3 path without Glue Crawler updates.

Term	Definition
Columnar Format	Parquet or ORC file formats — Athena reads only required columns, dramatically reducing scanned bytes (and cost).
CTAS (Create Table As Select)	Create a new table from query results — converts CSV to Parquet, saves to S3.
Federated Query	Query data sources OUTSIDE S3 (RDS, DynamoDB, Redshift, on-premises) using Lambda-based data source connectors.
Athena for Apache Spark	Run Apache Spark jobs interactively within Athena — serverless Spark without managing clusters.
Named Query	Saved SQL query — reuse and share queries across team members.

9.3 Architecture & Working

```
USER submits SQL query via Athena Console / JDBC / API
↓
ATHENA QUERY ENGINE (Presto/Trino — fully managed by AWS)
1. Parse SQL → Build logical query plan
2. Consult Glue Data Catalog for table schema, S3 location, partitions
3. Prune partitions (skip irrelevant S3 folders based on WHERE clause)
4. Distribute work across parallel workers (no visible cluster to you)
↓
AMAZON S3 (Data never moves)
Workers read only required columns from Parquet/ORC/CSV files in S3
Parallel reads from multiple S3 objects simultaneously
↓
QUERY RESULT written to S3 results bucket
↓
USER sees result set in console, or BI tool fetches via JDBC/ODBC
```

Cost Optimization Techniques

Technique	How It Reduces Cost	Example Saving
Partition your data	Athena skips entire S3 folders not matching the WHERE clause	Query for 'Jan 2024' skips 11 other months — 92% less data scanned
Use Parquet/ORC format	Columnar: read only queried columns; compressed: less bytes	10x less data scanned vs CSV for aggregation queries → 10x cheaper
Compress files	GZIP, Snappy, ZSTD compression reduces S3 read volume	2–5x compression ratio on text/JSON data
Use CTAS to convert to Parquet	One-time conversion of CSV to Parquet permanently reduces future query costs	Future queries cost 10x less after conversion
Limit SELECT columns	SELECT only needed columns — less data scanned with columnar formats	SELECT id, amount vs SELECT * — often 80% less scanned

9.4 Features

Feature	Description
Serverless	Zero infrastructure — Athena auto-scales query capacity. No cluster management.
Federated Queries	Query 25+ data sources (RDS, DynamoDB, Redshift, HBase, CloudWatch) from a single SQL statement.
CTAS	Create new tables from query results, automatically partitioned and in Parquet format for future efficiency.
Workgroup Cost Controls	Set per-query data scan limits, workgroup-level budgets, and automatic query cancellation.
Partition Projection	Define partition logic in table properties — no Glue Crawler needed to add new date partitions.
Named Queries	Save and share SQL queries across your organization for reuse and consistency.
Athena for Apache Spark	Run PySpark notebooks interactively on serverless Spark compute — no cluster needed.
Query Result Reuse	Cache query results for up to 7 days — identical queries return cached results instantly at no scan cost.
Lake Formation Integration	Athena enforces column-level and row-level Lake Formation permissions automatically.

9.5 Real-World Use Cases

Scenario	Details
Ad-Hoc Log Analysis	DevOps engineers query 1TB of VPC Flow Logs in S3 with SQL to diagnose network issues — costs ~$5.
Cost & Usage Reports	Finance team queries AWS Cost & Usage Reports (stored in S3) to analyze spending by service/team.
Data Lake Exploration	Data scientists explore raw data lake before building ETL pipelines — no upfront loading needed.
Clickstream Analysis	Analyze 90 days of user clickstream data (Parquet in S3) for funnel analysis — $2–$10 per query.
Compliance Reporting	Query CloudTrail audit logs to identify unauthorized API calls across all accounts.
Federated Query	JOIN current data in Redshift WITH historical data in S3 in a single SQL statement.

9.6 Practical Example — Query S3 Data

```
-- Create external table pointing to S3 Parquet data
CREATE EXTERNAL TABLE IF NOT EXISTS sales_db.transactions (
order_id STRING,
customer_id STRING,
amount DOUBLE,
product_cat STRING,
sale_date DATE
```

```
)
PARTITIONED BY (year INT, month INT)
STORED AS PARQUET
LOCATION 's3://my-data-lake/sales/transactions/'
TBLPROPERTIES ('parquet.compression'='SNAPPY');

-- Load partitions (or use Glue Crawler)
MSCK REPAIR TABLE sales_db.transactions;

-- Query: Top 5 categories by revenue in 2024
SELECT product_cat,
SUM(amount) AS total_revenue,
COUNT(*) AS order_count,
AVG(amount) AS avg_order_value
FROM sales_db.transactions
WHERE year = 2024
GROUP BY product_cat
ORDER BY total_revenue DESC
LIMIT 5;

-- CTAS: Convert CSV table to Parquet + partition by year/month
CREATE TABLE sales_db.transactions_parquet
WITH (format='PARQUET', parquet_compression='SNAPPY',
partitioned_by=ARRAY['year','month'],
external_location='s3://my-data-lake/sales/transactions_parquet/')
AS SELECT *, YEAR(sale_date) year, MONTH(sale_date) month
FROM sales_db.transactions_csv;
```

9.7 Integrations & Comparison

AWS Service	Integration with Athena
AWS Glue	Athena uses Glue Data Catalog for all table/schema definitions. Glue Crawlers auto-populate Athena tables.
Amazon S3	All Athena data lives in S3. Query results written back to a designated S3 bucket.
AWS Lake Formation	Enforce column-level and row-level security on all Athena queries.
Amazon QuickSight	QuickSight connects to Athena directly — creates dashboards from S3 data without ETL.
AWS Lambda	Athena Federated Query uses Lambda functions as data source connectors for non-S3 sources.
Amazon Redshift	Athena Federated Query can JOIN Athena (S3) tables with Redshift tables in one SQL.
Amazon CloudWatch	Query VPC Flow Logs, CloudTrail, and other AWS service logs exported to S3.

Athena vs Redshift vs EMR — Choose the Right Tool

Criterion	Amazon Athena	Amazon Redshift	Amazon EMR
Setup	Zero (serverless)	Minutes (managed cluster)	5–10 min (cluster launch)

Criterion	Amazon Athena	Amazon Redshift	Amazon EMR
Query Language	SQL (Presto/Trino)	SQL (PostgreSQL-compatible)	SQL + Spark/Python/Scala
Data Location	S3 only (+ federated)	Redshift storage + S3 (Spectrum)	S3 + HDFS
Cost Model	$5/TB scanned	Per node-hour (~$0.25+)	Per EC2 instance-hour
Performance	Good (columnar, partitioned)	Excellent (AQUA, MPP)	Good (parallel Spark)
Best For	Ad-hoc queries, data exploration	Frequent heavy analytics, BI	Complex ETL, ML, streaming
Concurrent Users	Unlimited (serverless scales)	Hundreds (with Concurrency Scaling)	Based on cluster size

Appendix

Master Comparison: All 9 AWS Analytics Services

At-a-Glance Summary

Service	Type	Key Strength	Cost Model	Best For
Amazon EMR	Managed Clusters	Hadoop/Spark at scale	Per EC2 hour	Complex big data, ML training
Lake Formation	Data Governance	Fine-grained access control	Per feature used	Data lake security & catalog
Amazon Redshift	Data Warehouse	MPP SQL analytics	Per node-hour	Regular heavy analytics, BI
Amazon Kinesis	Streaming	Real-time data pipelines	Per shard-hour / per GB	Live data processing
AWS Glue	Serverless ETL	ETL + Data Catalog	Per DPU-hour	Data preparation, cataloging
Amazon MSK	Managed Kafka	Open Kafka ecosystem	Per broker-hour	Event streaming, CDC, microservices
OpenSearch	Search & Analytics	Full-text search + log analytics	Per node-hour	Search, log monitoring, SIEM
QuickSight	BI & Visualization	Serverless dashboards + ML insights	Per session / per author	Business dashboards, embedded analytics
Amazon Athena	Serverless SQL	Ad-hoc SQL on S3	$5/TB scanned	Data exploration, one-off queries

When to Use Which Service — Decision Guide

Q: I need to process TBs of log data with custom Spark code → **Amazon EMR**

Q: I need to set up a secure data lake with column-level access → **AWS Lake Formation + Glue**

Q: I need a BI warehouse for daily SQL reports by 100 analysts → **Amazon Redshift**

Q: I need to process IoT sensor data in real time → **Kinesis Data Streams + Analytics**

Q: I need to convert CSV to Parquet and catalog everything → **AWS Glue ETL + Crawler**

Q: I have a Kafka-based microservices architecture → **Amazon MSK**

Q: I need to search product catalogs or analyze application logs → **OpenSearch Service**

Q: I need beautiful dashboards for business users → **Amazon QuickSight**

Q: I need to run ad-hoc SQL on existing S3 data cheaply → **Amazon Athena**

Typical End-to-End Analytics Architecture on AWS

DATA SOURCES

Databases (RDS) ■ SaaS APIs ■ IoT Devices ■ Mobile/Web Apps ■ On-Prem Systems

↓ Real-Time: Kinesis KDS/MSK ■■

```
↓ Batch: AWS Glue Crawlers + ETL Jobs ■
DATA LAKE (Amazon S3 + Lake Formation + Glue Data Catalog) ■
Raw Zone → Glue ETL → Curated Zone (Parquet, partitioned) ■
↓ Batch analytics ↓ Stream analytics ■
Amazon EMR Amazon Athena Kinesis Analytics (Flink) ■■■■■
(Complex ETL) (Ad-hoc SQL) (Real-time aggregation) ■
↓ ↓ ■
Amazon Redshift (Central Data Warehouse — regular BI analytics) ■
↓ ■
Amazon QuickSight (Dashboards + ML Insights for business users) ■
↓ ■
Amazon OpenSearch (Log monitoring + search + anomaly detection) ■■■■■■
```

End of AWS Analytics Complete Guide

All services explained with beginner-friendly depth, real-world use cases, and practical examples.